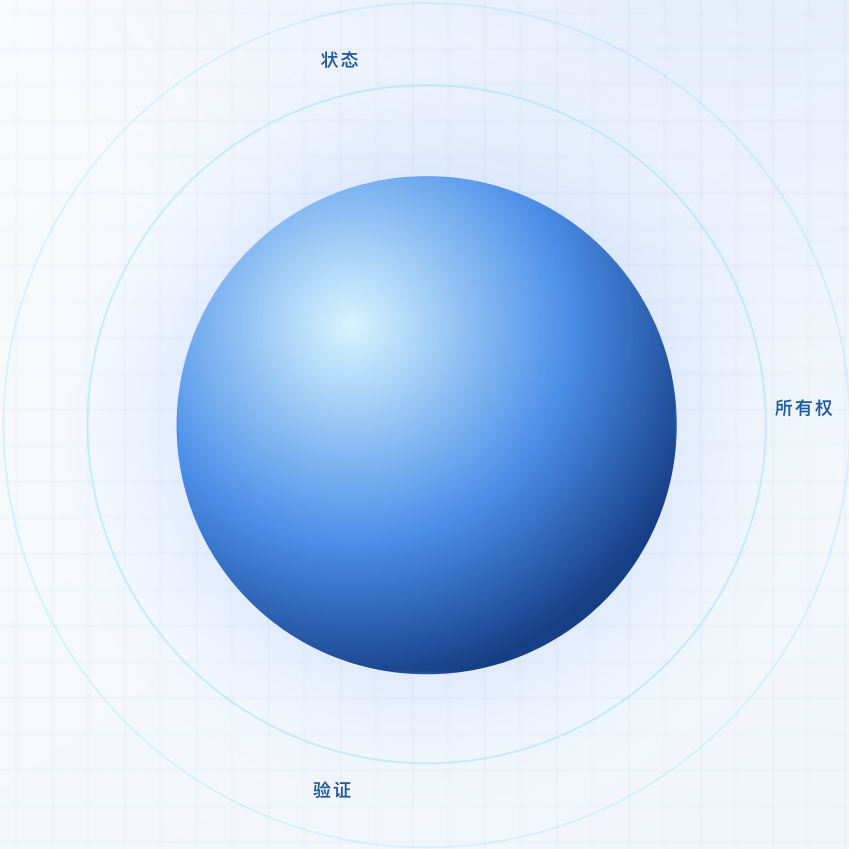
管理层架构简报 · TECHNICAL PREVIEW

ForgeFlow

面向 AI 辅助软件工程的有状态执行协议

位于交付方法之上的协议层：治理 AI Agent 如何决策、保存工程状态，以及恢复执行。

Agent-agnostic Core　仓库原生　证据驱动



为什么 AI Coding 需要
一套工程协议

Agent 已经可以生成代码、测试和重构。真正遗留的问题，是如何在长期交付中协调决策并保存可信状态。

AI 已经改善

速度与局部生产力

快速生成代码、解释逻辑、编写测试、执行重构和调用工具。

工程仍然需要

状态、授权与恢复

持久决策、受控 Scope、独立验证，以及回到正确上游的可靠路径。

01

上下文漂移

对话不是持久、可靠的项目事实来源。

02

决策混杂

Agent 静默代替业务、架构与交付角色作出决定。

03

Scope 扩张

单个任务在没有授权时扩大为无关实现。

04

恢复薄弱

失败被当作重试，而不是返回缺失决策的 Owner。

一套面向 AI Agent 的 工程执行协议

> Artifacts 保存状态。Workflows 拥有决策。Artifact Graph 授权执行。Handoffs 报告经过验证的执行转移。

定位

协议 + 参考实现

仓库原生 Markdown 契约，配合确定性解析、状态计算和可选编排。

与 AGENT 的关系

Agent-agnostic Core

Core 不依赖模型 API 或厂商 Runtime；Adapter 隔离宿主专属调用。

当前集成边界

Codex 与 Copilot

当前提供专用 Launcher；其他 Agent 可读取协议，但不宣称已经完成验证。

管理层结论：ForgeFlow 治理 Agent 如何参与交付，但不替代工程判断、CI、源代码管理和人的最终责任。

ForgeFlow 不替代 SDLC，
而是治理 AI 如何参与其中。

SDLC 描述软件交付阶段和组织实践；ForgeFlow 定义 Agent 在这些阶段中的授权、证据、转移与恢复规则。

传统 SDLC

软件交付如何组织

规划、需求、设计、实现、测试、发布与运维，并由组织选择适合的方法。

过程模型

FORGEFLOW PROTOCOL

AI 执行如何被授权

谁可以决定、哪些证据有效、下一步允许执行什么，以及受阻时必须返回哪里。

执行控制模型

SDLC 提供组织级生命周期，ForgeFlow 提供 AI Agent 在其中运行的有状态执行协议。

定位：ForgeFlow 是位于具体 SDLC 方法之上的协议基础设施，不是新的瀑布模型、Agile 变体或工程管理替代品。

Workflow / Artifact / Handoff / Ownership

Workflow、Artifact 与 Handoff 的三机制，由 Artifact Graph 补全：它负责判断现在真正允许执行什么。

01

Workflow

为某一类工程决策指定唯一责任主体。

谁可以决定?

02

Artifact

保存已批准的意图、需求、设计、实现和 Review 证据。

什么是真实状态?

03

Handoff

报告跨 Workflow 或 Scope 的已验证转移。

谁来接手?

04

Artifact Graph

验证版本、依赖与 Blocker，然后解析下一授权。

现在允许做什么?

Handoff 本身不会创造事实。每个参与者都必须从仓库证据重新构建 Graph，并再次解析授权。

由协议定义、带显式门禁的执行序列

PLAN
项目意图
Feature 顺序

›

GRILL
可观察行为
业务澄清

›

SOLUTION
工程基线
跨 Feature 设计

›

SLICE
有限交付
有序 Slice

›

TDD
实现
测试与证据

›

REVIEW
独立
PASS 或 FAIL

业务门禁

Planning + Requirements

Feature 和可观察结果必须有效，才进入工程设计。

工程门禁

Solution + Slicing

架构与交付边界必须有效，才进入实现。

质量门禁

TDD + Review

每个 Slice Attempt 的实现证据必须经过独立 Review。

这是 ForgeFlow Workflow 模型，不是对完整 SDLC 的替代。证据暴露上游缺口时，Return 与 Recovery 是协议级路径。

两个维度的受控迭代

项目同时按产品价值和实现风险拆分，而不是把两者压平成一张无序任务清单。





确定性完成：Planning 决定 Feature 顺序，Slice Plan 决定 Slice 顺序。单个 Slice 成功不会被误判为项目完成。

每一种决策只有一个权威 Owner

所有权机制解决 AI 因为“能决定”就把所有问题都在当前阶段自行决定的问题。

PLAN	项目意图、Feature 顺序、排除项和增量 Change Set	PROJECT
GRILL	业务行为、验收标准和未解决歧义	FEATURE
SOLUTION	架构、接口、数据、安全、运维与验证策略	PROJECT
SLICE	有限实现切片及其声明执行顺序	FEATURE
TDD	单个获准 Slice Attempt 的实现与测试	SLICE
REVIEW	独立 PASS/FAIL 决策和 Findings 路由	SLICE ATTEMPT

失败返回缺失决策的 Owner

RETURN 路由

Business Blocker	返回 Grill；Project Scope 返回 Plan
Engineering Blocker	返回 Solution
Slice Boundary	返回 Slice
Implementation Issue	返回 TDD

无需对话记忆的 RECOVERY

从证据重新启动

新进程重建 Artifact Graph，验证当前持久化 Blocker，只重新授权正确 Owner 与 Scope。

Runtime History 用于诊断，仓库证据才是恢复权威。

ForgeFlow 区分“重试执行”和“修正导致执行无法继续的决策”。

工程知识沉淀为版本化证据链

Planning

Intent、Feature 顺序、约束、Change Set

Requirement

行为、验收标准、业务 Contract Version

Solution Plan

工程基线和每 Feature Engineering Version

Slice Plan

单个 Feature 的有序交付边界

TDD Record

实现 Attempt、测试和当前上游版本

Review Report

不可变 Attempt、Findings、PASS/FAIL 和 Return

当前事实

只有规范路径和有效当前版本参与授权。

历史追踪

被替换 Contract 与补充 Report 进入专用 History Root。

作用域化变更

Feature Contract 与 Engineering Version 保留未受影响证据。

治理嵌入执行过程，而不是事后补充

Scope	Project、Feature、Slice 类型化授权，防止静默扩张。
Decision	Artifact 独占所有权，防止 Agent 自行批准假设。
Quality	TDD 证据和独立 Review 形成逐 Attempt 质量门禁。
Traceability	Version Edge 连接意图、需求、设计、实现和验证。
AI 越权	非法、过期或矛盾证据失败关闭，不授权生产性工作。
Extensibility	Markdown Review Plugin 增加检查，但不转移 PASS/FAIL 所有权。

重要边界：ForgeFlow 降低流程风险，但不保证模型正确，也不替代安全控制、分支保护、CI 和人工 Review。

设计上与 Copilot / Claude / Codex 无关

ForgeFlow 的协议事实保存在仓库契约中，而不是绑定在某个模型、Prompt 产品或厂商 Runtime 中。

1

稳定 Core

Workflow、Artifact、Handoff 与 Ownership 语义保持宿主无关。

2

轻量 Adapter

Launcher 把宿主调用转换为同一套协议定义的 Workflow 入口。

3

Agent 可替换

团队更换 Agent 时，无需替换工程状态与治理契约。

当前已验证

当前为 Codex 与 GitHub Copilot 提供 ForgeFlow 专用 Launcher 和 Adapter 内容。

协议兼容方向

Claude 和其他 Agent 可通过符合协议的 Adapter 接入，但本文不把它们表述为当前已验证集成。

Agent Independence 保护的是长期投入：工程证据、决策所有权与转移规则可以持续复用。

从 AI Coding Assistance 走向可治理 AI Engineering

战略方向是构建可移植执行协议，让 AI 在整个软件生命周期中的参与可观察、可替换、可问责。

强化 CORE

稳定协议与测试矩阵

冻结版本化契约，扩展 Transition 覆盖，在不同实现中保持确定性行为。

方向，不是当前能力声明

扩展边缘

更多经过验证的 Adapter

只有验证真实发现、调用、权限和验收行为后，才增加宿主集成。

方向，不是当前能力声明

治理运营化

更安全的自动化与升级路径

改善分发、刷新、证据检查和受控自动化，不创建第二份事实来源。

方向，不是当前能力声明

> 从 AI Coding 走向 Protocol-Driven AI Engineering：长期资产不是某一个更聪明的 Agent，而是一套能够安全协调不断变化 Agent 的工程系统。